

Inventory Reservation System

1. Project Overview

The Inventory Reservation System is a full-stack web application designed to manage inventory reservations across multiple warehouses in real time.

The system allows users to:

- View products and warehouse inventory
- Reserve products temporarily
- Confirm reservations
- Release reservations
- Automatically expire unused reservations
- Prevent overselling using atomic database transactions

This project was developed using modern technologies including:

- Next.js
- TypeScript
- Prisma ORM
- PostgreSQL (Supabase)
- Vercel Deployment

The project focuses heavily on:

- concurrency handling
 - transactional integrity
 - inventory consistency
 - real-time reservation management
-

2. Problem Statement

In inventory systems, multiple users may attempt to reserve the same product simultaneously.

Without proper handling:

- stock may become negative
- duplicate reservations may occur
- overselling can happen
- data inconsistency may occur

This system solves these issues using:

- database transactions
 - stock validation
 - reservation expiry logic
 - atomic updates
-

3. System Architecture

Frontend

- Next.js App Router
- React Hooks
- Tailwind CSS
- Client-side state management

Backend

- Next.js API Routes
- Prisma ORM
- PostgreSQL database

Database

- Supabase PostgreSQL

Deployment

- Vercel
-

4. Technologies Used

Technology	Purpose
Next.js	Full-stack framework

TypeScript	Type safety
Prisma ORM	Database operations
PostgreSQL	Relational database
Supabase	Cloud database hosting
Tailwind CSS	UI styling
Vercel	Deployment
Node-Cron	Scheduled cleanup tasks

5. Database Design

The system uses four primary tables.

5.1 Product Table

Stores product details.

Fields

- id
- name
- description
- createdAt

Purpose

Represents products available for reservation.

5.2 Warehouse Table

Stores warehouse details.

Fields

- id
- name
- location
- createdAt

Purpose

Represents physical warehouse locations.

5.3 Inventory Table

Stores inventory quantities.

Fields

- id
- productId
- warehouseId
- totalStock
- reservedStock

Purpose

Tracks:

- actual stock
- temporarily reserved stock

Why reservedStock was implemented

Using only totalStock is unsafe because:

- multiple users may reserve simultaneously
- temporary holds must be tracked separately

reservedStock helps:

- avoid overselling
 - maintain consistency
 - support temporary reservations
-

5.4 Reservation Table

Stores reservation records.

Fields

- id
- productId
- warehouseId
- quantity
- status
- expiresAt
- createdAt

Reservation Status Values

- PENDING
- CONFIRMED
- RELEASED
- EXPIRED

Purpose

Tracks reservation lifecycle.

6. Reservation Workflow

Step 1 – User Views Products

Frontend fetches product and inventory information from:

/api/products

The user can view:

- product name
- warehouse
- available stock

Available stock is calculated using:

`availableStock = totalStock - reservedStock`

Step 2 – User Creates Reservation

User clicks:

Reserve Product

API called:

`POST /api/reservations`

What happens internally

1. Inventory row is fetched
 2. Available stock is calculated
 3. Stock validation is performed
 4. Database transaction starts
 5. `reservedStock` increases
 6. Reservation row is created
 7. Expiry timestamp is generated
-

7. Why Transactions Were Used

Database transactions are critical in inventory systems.

Without transactions:

- two users may reserve the same item simultaneously
- inconsistent inventory may occur
- duplicate stock allocation may happen

The project uses:

`prisma.$transaction()`

This guarantees:

- atomic operations
- consistency
- rollback on failure

Example

If inventory update succeeds but reservation creation fails:

- transaction automatically rolls back
 - inventory remains correct
-

8. Confirm Reservation Flow

API:

POST /api/reservations/[id]/confirm

What happens

1. Reservation is fetched
 2. System checks:
 - reservation exists
 - status is PENDING
 - reservation is not expired
 3. Transaction begins
 4. totalStock decreases
 5. reservedStock decreases
 6. Reservation status becomes CONFIRMED
-

9. Release Reservation Flow

API:

POST /api/reservations/[id]/release

Purpose

Allows user to cancel reservation manually.

What happens

1. Reservation is validated
2. Transaction starts
3. reservedStock decreases
4. Reservation status becomes RELEASED

Why this is important

Without release functionality:

- inventory may remain locked
 - unused stock becomes unavailable
-

10. Automatic Reservation Expiry

Reservations are temporary.

If users do not confirm within the expiry window:

- reservation expires automatically
 - stock becomes available again
-

Expiry Duration

The project currently uses:

3 minutes

Why 3 minutes was chosen

- realistic demo timing
- easier testing
- prevents long inventory blocking

The duration was configurable and can easily be changed.

Expiry Cleanup Logic

A scheduled cleanup task checks for expired reservations.

Expired reservations:

- change status to EXPIRED
- reservedStock decreases

Why cleanup was implemented

Without cleanup:

- expired reservations still block stock
 - inventory becomes unusable over time
-

11. Pagination Implementation

Reservation history API supports pagination.

Example:

`/api/reservations?page=1&limit=2`

Why pagination was implemented

Without pagination:

- large responses increase loading time
- performance decreases
- frontend rendering becomes inefficient

Pagination improves:

- scalability
 - performance
 - response efficiency
-

12. Frontend Features

Features Implemented

- Product cards
- Warehouse inventory display
- Reservation timer
- Reservation history
- Loading states
- Error handling
- Auto refresh
- Responsive UI
- Stock display

13. Real-Time UI Updates

Frontend automatically refreshes data after:

- reservation creation
- confirmation
- release

Why this was implemented

Without refreshing:

- UI may show stale stock
 - user sees incorrect inventory
-

14. Countdown Timer

Each reservation displays:

- minutes
- seconds

Example:

2m 45s

Why this was implemented

This improves:

- user experience
 - reservation visibility
 - urgency awareness
-

15. Edge Cases Handled

15.1 Insufficient Stock

Scenario

User tries to reserve more stock than available.

Handling

API returns:

409 Conflict

Why

Prevents overselling.

15.2 Duplicate Confirmation

Scenario

User confirms already confirmed reservation.

Handling

System validates status before update.

15.3 Expired Reservation Confirmation

Scenario

User tries confirming expired reservation.

Handling

Request rejected.

15.4 Reservation Release After Expiry

Handling

Validation prevents invalid state transitions.

15.5 Concurrent Reservations

Scenario

Two users reserve same stock simultaneously.

Handling

Transactions maintain consistency.

15.6 Stock Reaches Zero

Scenario

All inventory gets sold.

Current System Behavior

- reservation requests fail
- API returns insufficient stock

Why this behavior is correct

In real inventory systems:

- sold stock does not automatically refill
- warehouses manually restock inventory

How stock can be updated

Option 1 – Prisma Studio

Developer can manually edit:

- totalStock
- reservedStock

Option 2 – Admin Restock API

A future enhancement could include:

POST /api/restock

This would allow warehouse/admin users to refill inventory.

Why auto-refill was NOT implemented

Automatic refill is unrealistic because:

- physical inventory cannot regenerate automatically
 - inventory should reflect actual warehouse stock
-

16. API Endpoints

Method	Endpoint	Purpose
GET	/api/products	Fetch products
POST	/api/reservations	Create reservation
GET	/api/reservations	Fetch reservations
POST	/api/reservations/[id]/confirm	Confirm reservation
POST	/api/reservations/[id]/release	Release reservation

17. UI Design Decisions

Colors

Different colors indicate:

- active reservation
- stock status
- confirmation
- release action

Why color coding was used

Improves:

- readability
 - user interaction
 - action clarity
-

Responsive Layout

The UI adjusts for:

- desktop
- tablet
- mobile

Why responsiveness was implemented

Modern applications must support multiple screen sizes.

18. Deployment

Frontend Hosting

Vercel

Database Hosting

Supabase PostgreSQL

Deployment Challenges Faced

Issue

Frontend used:

<http://localhost:3000>

Problem

This only works locally.

Solution

Replaced with relative API routes:

/api/products

/api/reservations

Database Connectivity Issue

Issue

Supabase direct connection used IPv6.

Problem

Vercel required IPv4-compatible pooling.

Solution

Used Supabase pooler connection string.

19. Security and Validation

Implemented validations include:

- required field validation
- stock validation
- reservation status validation
- expiry validation

Why validation is important

Prevents:

- invalid data
- corrupted inventory
- illegal state changes

20. Future Enhancements

Potential future improvements:

Authentication

- Admin login
- User accounts

Role-Based Access

- Admin
- Warehouse manager
- Customer

Real-Time Updates

- WebSockets
- Live inventory sync

Analytics Dashboard

- Reservation trends
- Warehouse statistics
- Sales tracking

Restock API

- Add inventory dynamically

Email Notifications

- Reservation confirmation
 - Expiry alerts
-

21. Key Learning Outcomes

This project provided practical experience with:

- database transactions
 - concurrency handling
 - Prisma ORM
 - REST API design
 - Next.js full-stack development
 - PostgreSQL integration
 - deployment debugging
 - production environment configuration
 - frontend/backend integration
-

22. Conclusion

The Inventory Reservation System successfully demonstrates:

- reliable inventory reservation
- atomic stock handling
- transaction-based consistency
- scalable backend architecture
- responsive frontend design
- production deployment readiness

The system effectively prevents overselling while maintaining accurate inventory states across multiple warehouses.

The project reflects real-world backend engineering concepts including:

- reservation lifecycle management
- concurrency control
- expiry handling
- transactional integrity
- scalable API design

Overall, the application provides a strong foundation for enterprise-level inventory and warehouse management systems.